

TOPIC - HTML5 (APIs) - Audio, Video and Progress

1) What happens to the content between `<video>` and `</video>` if a browser does not support the `<video>` tag?

Explain with reference to “fallback content.”

2) Write an HTML snippet to embed an audio file named `example.mp3` on a web page.

- The player should include browser controls.
- Add fallback text for unsupported browsers.

3) Write HTML code for a progress bar that shows a task is 75% complete. Label it as “Upload Progress”.

TOPIC - Geolocation

1) Write a JavaScript function using the Geolocation API that displays both the user’s latitude and longitude in an alert box when a button is clicked. Show the HTML and JS required.

2) Explain the purpose of the three parameters that can be passed to `navigator.geolocation.getCurrentPosition()`. Give an example call using all three and briefly describe what each parameter does.

3) Compare `getCurrentPosition()` and `watchPosition()`. List two practical scenarios for each method, explaining why that method is more appropriate in each case.

TOPIC - Callbacks & Promises

1) Explain with an example what “callback hell” is. Why can it be a problem in asynchronous JavaScript?

How do Promises help solve the issues caused by callback hell?

2) Write a complete JavaScript function `delayedGreeting(name, delay, callback)` that takes a name, a delay in milliseconds, and a callback function. It should print `"Hello, <name>!"` after the specified delay, and then call the callback function to print `"Callback complete."`

Provide sample usage and expected output.

3) Consider this Promise code:

```
const result = new Promise((resolve, reject) => {
  let success = Math.random() > 0.5; // Random true or false
  if (success) {
    resolve("Operation successful");
  } else {
    reject("Operation failed");
  }
});

result
  .then(msg => console.log("Resolved: " + msg))
  .catch(err => console.log("Rejected: " + err));
```

- a) What are the possible outputs?
- b) Briefly describe what the `.then()` and `.catch()` methods do in the context of a Promise.

TOPIC - XML Vs JSON

- 1) Given the following JavaScript code:

```
javascript
```

```
let car = { brand: "Toyota", year: 2022, electric: false };
console.log(JSON.stringify(car));
```

- a) What will be printed in the console?
- b) What is the purpose of `JSON.stringify` here?

- 2) Consider the XML string:

```
<user>
```

```
<name>Alice</name>
<age>25</age>
</user>
```

a) Write JavaScript code using `DOMParser` to parse this XML string into an XML DOM object.

b) Show how to access and print the `<name>` value from the parsed XML.

3) You receive data from two APIs, one as JSON and the other as XML.

a) With one line each, write how you would parse the JSON string `jsonStr` and the XML string `xmlStr` into JavaScript-accessible objects.

b) Name 2 advantages of JSON over XML for web application data exchange, based on the properties described in the slides.

TOPIC - Single Page Application

1) Explain what a Single Page Application (SPA) is. How does it differ from a traditional Multi-Page Application (MPA) in terms of page loading, user experience, and SEO?

2) What are some common challenges developers face when building SPAs? Explain how server-side rendering (SSR) or pre-rendering can help address one of these challenges.

TOPIC - Async/Await

1) Explain what the `async` keyword does when placed before a function declaration. What kind of value does the function return?

2) What happens if an awaited Promise rejects and the code is not wrapped in a try/catch block? How should errors be handled in async functions to avoid unhandled rejections?

3)

```
async function fetchUser(userId) {  
  try {  
    const response = await  
fetch(`https://jsonplaceholder.typicode.com/users/${userId}`);  
    const user = await response.json();  
    console.log(user);  
  } catch (error) {  
    console.error("Failed to fetch user:", error);  
  }  
}  
  
fetchUser(1);
```

Explain the role of the `try/catch` block in the above async function. What would happen if the `try/catch` was omitted and the fetch failed?

TOPIC - AXIOS

1) What common configuration options does Axios support in requests? Explain the use of `headers`, `params`, and `timeout` options in Axios.

2) Write an Axios GET request to fetch user data from `'https://jsonplaceholder.typicode.com/users/1'`. Log the user's name to the console, and handle errors by logging an error message. Use `async/await` syntax.

TOPIC - ReactJS - MERN Introduction

1) Explain the role of each component in the MERN stack (MongoDB, Express.js, React.js, Node.js) using the restaurant analogy given in the presentation.

TOPIC - JSX, rendering of Elements

1) Write a React functional component named `UserGreeting` that accepts a `username` prop and displays "Welcome, [username]!" in an `<h1>` element. Also, demonstrate how you would use this component inside a parent component passing a sample username.

2) What is JSX in React, and how does it differ from regular HTML? Explain the role of Babel in handling JSX code.

3) Write a React component using JSX that renders an unordered list with three tasks: "Debugging code," "Writing documentation," and "Presenting talks." Ensure the JSX syntax rules are correctly followed.

TOPIC - React Component Styling

1) `const element = <h1>Hello, {2 + 2}!</h1>;`

Question:

What will be rendered on the screen by this JSX code? Explain how JavaScript expressions are embedded within JSX and how they are evaluated.

```
2) function TaskList() {  
  return (  
    <>  
      <h2>Tasks</h2>  
      <ul>  
        <li>Debugging code</li>
```

```

        <li>Writing documentation</li>
        <li>Presenting talks</li>
    </ul>
</>
);
}

```

Question:

Why is it necessary to wrap the `<h2>` and `<ul>` elements inside a single enclosing tag or React fragment in this React component? What JSX syntax rule does this demonstrate?

```

3) const users = [
  { id: 1, name: 'Alice', online: true },
  { id: 2, name: 'Bob', online: false },
];

const userList = (
  <ul>
    {users.map((user) => (
      <li key={user.id} style={{ color: user.online ? 'green' :
'gray' }}>
        {user.name} {user.online ? '(Online)' : '(Offline)'}
      </li>
    ))}
  </ul>
);

```

Question:

Explain why the `key` property is used in the list items. What will be the color and text shown for each user in this rendered list?

```
4) function MyForm() {  
  const [name, setName] = React.useState('');  
  return (  
    <input  
      type="text"  
      value={name}  
      onChange={(e) => setName(e.target.value)}  
    />  
  );  
}
```

Question:

What is the purpose of the `value` and `onChange` attributes in this input element? How does React control the form input in this component?